ITA03-MOBILE COMPUTING
ASSIGNMENT-5

ASSIGNMENT 5 – CO5

CO5 Coverage: Six Characteristics of mobile app - connectivity, convenience, localization, reachability, security and personalization - Android Intents and Services - Synchronization and Replication of Mobile Data - Android & IOS multimedia framework (Core Video framework and Core Media framework) -Peer- to Peer Architecture for mobile application -Performance and Multithreading -Graphics and UI Performance.

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

Course Outcome – CO5: Create mobile application design using diverse mobile operating system frameworks to ensure performance efficiency, security, usability, and scalability.

Nature: Individual

Duration: One Week **Total Marks:** 50

Assessment Components

PART A – Design, Analysis & Pseudocode / Architectural Reasoning: 25 Marks

PART B – Implementation, Instrumentation & Computational Investigation: 25 Marks

Submission Date:12/11/2026

PURPOSE OF THE ASSIGNMENT

Modern mobile applications must deliver responsive user experiences while operating under severe constraints of battery, memory, network variability and security threats.

This assignment investigates a fundamental engineering question:

How should a mobile application engineer design, implement and evaluate components so that the application remains efficient, secure, usable and scalable when the number of concurrent users, data volume and device heterogeneity grows?

You are expected to demonstrate the following progression:

Model → Design Intent/Service → Synchronise → Optimise → Measure → Analyse Trade-offs → Approximate / Heuristic → Evaluate

Simply obtaining a working application is not sufficient. Your architectural reasoning, performance instrumentation, experimental evidence and interpretation are all part of the assessment.

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

25 Marks

All designs and pseudocode must follow a clear, language-independent algorithmic / architectural notation.
Implement the constraint-aware background design from Task A1 (or a closely related sensorupload scenario).

Your program must record:

- Number of work items enqueued
- Number of work items deferred / cancelled by constraints
- Number of successful uploads / executions
- Battery / network state at each decision point (simulated or real) ●

Maximum concurrent workers

- Final outcome (all data delivered or residual backlog)

B1.1 – Validate the Implementation | 2 Marks

Test the implementation using at least:

● One scenario that succeeds under good connectivity and battery ● One scenario

that must defer or cancel work under Doze / low-battery ● One boundary case

(permission revoked mid-execution) analyse

what each test is intended to verify.

B1.2 – Compare Design with Implementation | 1 Mark

Compare your implementation with your Part A pseudocode. Identify any modification made during implementation and analyse why it was necessary.

B1.3 – Improve the Scheduling Order | 2 Marks Implement

two versions:

Version 1: Process work items in arrival / FIFO order.

Version 2: Use a reasoned priority heuristic intended to surface critical or high-value work earlier while respecting constraints.

You must formulate and analyse your own selection rule.

B1.4 – Measure the Effect | 2 Marks For
the same workloads compare:

Measure	**Version 1**	**Version 2**
Work items enqueued		
Deferred / cancelled		
Successful executions		
Peak concurrent workers		

Did your ordering rule reduce wasted work or improve delivery of critical data? Why or why not?

Evidence Expected: Source code, test suite, instrumentation data, two scheduling strategies and experimental interpretation.

ANSWER – A1 TASK A1 – Design a Constraint-Aware Intent & Service Architecture (7 Marks) A1.1 – Model the Problem

Element	Description
Input	Sensor readings, timestamps, connectivity state, battery level, Doze state, permission status, user preferences and authentication/logout events.
Output	Encrypted sensor records uploaded to the backend, delivery status and critical-event notifications.
Decision at each stage	Collect or skip a reading; queue locally or upload; select WorkManager or an appropriate foreground service; defer, retry or cancel; notify or suppress notification.
Constraints	Battery and Doze limits, network availability, user permission, authenticated session, encryption, background execution rules and responsiveness.
Goal condition	All eligible records are delivered once or remain safely queued; critical events are notified promptly; no unauthorised data is exposed.

A1.2 – Runtime State

Maintain a state record for each work item: task ID, type, creation time, priority, encrypted payload reference, retry count, network requirement, battery threshold, permission status, authentication status, current state and last error. Global state includes network type/quality, battery level, Doze status, current user/session and maximum permitted concurrency.

A1.3 – Pruning / Feasibility Conditions

- Defer non-urgent uploads when the network is unavailable, metered against user preference, or too weak for the payload.
- Defer non-critical work when battery is below the configured threshold or the device is in Doze; use constraint-aware scheduled work.
- Reject a task if permission is revoked, the user is logged out, the payload cannot be encrypted, or authentication has expired.

- Cancel duplicate work items using a unique task ID; do not delete the only local copy until server acknowledgement is verified.
- Critical alerts may use a permitted urgent notification path, but must still pass authentication, privacy and platform-policy checks.

A1.4 – Architectural Pseudocode

PROCEDURE SCHEDULE_OR_EXECUTE(queue, systemState)\n WHILE queue is not empty DO\n item ← SELECT highest-priority eligible item\n IF item.permissionValid = FALSE OR item.sessionValid = FALSE THEN\n CANCEL item; record reason; CONTINUE\n END IF\n IF item.sensitive AND item.encrypted = FALSE THEN\n REJECT item; record security error; CONTINUE\n END IF\n IF item.requiresNetwork AND systemState.networkAvailable = FALSE THEN\n DEFER item; enqueue WorkManager retry; CONTINUE\n END IF\n IF systemState.doze AND item.priority != CRITICAL THEN\n DEFER item until allowed execution window; CONTINUE\n END IF\n IF systemState.battery < item.minimumBattery AND item.priority != CRITICAL THEN\n DEFER item; CONTINUE\n END IF\n intent ← BUILD explicit Intent for internal component\n worker ← CREATE constrained worker for upload or collection\n result ← EXECUTE worker(item)\n IF result = ACKNOWLEDGED THEN\n MARK delivered; remove local payload after safe acknowledgement\n ELSE IF result = RETRYABLE_FAILURE THEN\n INCREMENT retryCount; retain encrypted local copy; schedule backoff\n ELSE\n CANCEL or quarantine item; log non-sensitive error\n END IF\n END WHILE\nEND PROCEDURE

A1.5 – Trace the Design

Event	Constraint check	Action	Result
1. Sensor reading received	Permission valid; session active	Collect and encrypt reading	Queued safely
2. Network disconnects	Upload requires network	Defer upload; retain local record	No data loss
3. Device enters Doze	Non-critical work restricted	WorkManager waits for eligible window	Battery protected
4. Critical threshold crossed	Permission and notification channel valid	Create high-priority alert and notify through allowed path	User alerted
5. User logs out	Session becomes invalid	Cancel pending authenticated uploads; retain only policypermitted encrypted records or securely clear them	No upload using stale credentials

Conclusion: The design separates collection, storage, upload and notification. Feasibility checks occur before execution, while retry and cancellation rules preserve privacy and prevent loss of unacknowledged data.

TASK A2 – DON'T SYNCHRONISE WHAT CANNOT BE SAFE

TASK A1 – A1 – Architectural Responsibilities and Component Boundaries

Separation of responsibilities

A reliable mobile architecture should separate sensor acquisition, local persistence, scheduling, network transfer and user notification. The sensor component should only collect a reading when the required permission and device capability are available. The persistence component should store the reading with a stable identifier and timestamp. The scheduler should decide when the item is eligible for execution, while the upload worker should perform the network operation and report a result. This separation prevents a temporary network failure from interrupting sensor collection and prevents a user-interface screen from becoming responsible for long-running work. Each component has a clear input, output and failure boundary.

Intent selection and component exposure

An explicit Android Intent identifies the exact internal component that should receive a request. It is preferable for sensitive internal operations because the destination is known and the request is not broadcast to unrelated applications. The Intent should carry only the minimum information required, such as a work identifier or a reference to an encrypted local record. Large sensor payloads should remain in protected storage rather than being copied into Intent extras. Exported components should be avoided unless an external application genuinely needs access; when exposure is required, permissions and input validation must be applied.

Service and worker choice

A foreground service is appropriate only when the user is actively aware of an operation that must continue immediately, such as a user-started live recording or navigation session. It must display the required ongoing notification and comply with current Android foreground-service restrictions. Deferrable uploads and retryable maintenance are better represented by WorkManager because the operating system can schedule them around network, charging and idle constraints. The architecture should not keep a service alive merely to avoid designing a proper queue.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Component boundary	Keep sensor reading, local storage, scheduling and upload responsibilities separate.	A failure in one stage should not corrupt the other stages.
Intent and service	Use explicit intents for internal components; use a foreground service only for user-visible ongoing work.	Limits accidental exposure and respects Android background rules.

Eligibility check	Check permission, session, encryption, network, Doze and battery before starting work.	Prevents unsafe or wasteful execution.
Recovery	Persist state, retain unacknowledged records and retry with bounded backoff.	Supports process restarts without losing data or creating duplicates.

ENGINEERING TAKEAWAY

Key point: a task is eligible only after security, permission and resource checks pass. Keep unacknowledged data safely stored.

TASK A1 – A1 – State Management, Eligibility and Security

Work-item state model

Each work item should move through a controlled set of states such as CREATED, QUEUED, ELIGIBLE, RUNNING, DEFERRED, DELIVERED, CANCELLED and QUARANTINED. A state transition should be recorded only after the corresponding action has been confirmed. For example, an item must not be marked DELIVERED merely because an upload request was sent; it becomes delivered only after the backend acknowledges the stable record identifier. Recording transitions makes recovery possible after a process crash and allows the implementation to distinguish a pending item from a permanently rejected item.

Eligibility checks at execution time

Conditions should be checked both when work is scheduled and immediately before it begins. Battery level, network connectivity, Doze state and permissions can change between those moments. A worker that was eligible when queued may no longer be eligible when the operating system starts it. The worker should therefore re-read the current state, apply the same safety rules and defer without losing the local record when a temporary constraint is not satisfied. Rechecking conditions avoids relying on stale assumptions and makes the design robust to unpredictable mobile-device behaviour.

Security and identity controls

Authentication and permission status are separate checks. A user may still have granted a sensor permission while the application session has expired, or may remain signed in after revoking access to a particular sensor. Both conditions must be evaluated. Sensitive readings should be encrypted before they enter a transfer queue, and encryption keys should be protected by platform security facilities. Logs should record task identifiers, result categories and timing, but should not contain

raw sensor values, access tokens or personal information. A logout event should invalidate credentials and prevent queued work from using an old session.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Component boundary	Keep sensor reading, local storage, scheduling and upload responsibilities separate.	A failure in one stage should not corrupt the other stages.
Intent and service	Use explicit intents for internal components; use a foreground service only for user-visible ongoing work.	Limits accidental exposure and respects Android background rules.
Eligibility check	Check permission, session, encryption, network, Doze and battery before starting work.	Prevents unsafe or wasteful execution.
Recovery	Persist state, retain unacknowledged records and retry with bounded backoff.	Supports process restarts without losing data or creating duplicates.

ENGINEERING TAKEAWAY

Key point: a task is eligible only after security, permission and resource checks pass. Keep unacknowledged data safely stored.

TASK A1 – A1 – Failure Recovery, Instrumentation and Evaluation

Retry and backoff policy

A retry should be scheduled only when the failure is temporary and the operation remains authorised. Network timeouts, temporary server unavailability and rate limiting may justify a delayed retry. Invalid credentials, revoked permission and malformed payloads require a different response because repeating the same request cannot correct the underlying problem. Exponential backoff with a maximum delay reduces repeated radio activation and avoids overwhelming the backend. A small random jitter can prevent many devices from retrying at exactly the same moment after a shared outage.

Acknowledgement and local-data retention

The local encrypted record should be retained until the server confirms successful receipt using a stable identifier or idempotency key. If the device loses connectivity after the server accepts a record but before the acknowledgement reaches the device, the next attempt may repeat the request. Idempotent handling allows the server to recognise the duplicate and return the original result rather

than creating a second record. Only after the acknowledgement is verified should the local copy be removed according to the retention policy. This is essential to avoid data loss during partial failures.

Measurements for evaluating the architecture

The implementation should count enqueued, started, completed, deferred, cancelled and quarantined work items. It should also record queue age, retry count, execution duration, network type, battery state at the decision point and peak concurrent workers. These measurements explain whether the scheduler is reducing unnecessary work or merely postponing it. A successful design should deliver eligible records, preserve records that cannot yet be delivered, avoid unauthorised transfer and keep concurrency within the selected limit. Results should be reported with the workload assumptions so that illustrative simulation values are not mistaken for measurements from a physical device.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Component boundary	Keep sensor reading, local storage, scheduling and upload responsibilities separate.	A failure in one stage should not corrupt the other stages.
Intent and service	Use explicit intents for internal components; use a foreground service only for user-visible ongoing work.	Limits accidental exposure and respects Android background rules.
Eligibility check	Check permission, session, encryption, network, Doze and battery before starting work.	Prevents unsafe or wasteful execution.
Recovery	Persist state, retain unacknowledged records and retry with bounded backoff.	Supports process restarts without losing data or creating duplicates.

ENGINEERING TAKEAWAY

Key point: a task is eligible only after security, permission and resource checks pass. Keep unacknowledged data safely stored.

Improve a Weak Data-Replication Design

5 Marks

Consider the following naïve approach for synchronising mobile data with a cloud backend:

ALGORITHM SyncAll(LocalStore, RemoteAPI)

FOR each record R in LocalStore DO

Upload R to RemoteAPI

IF network fails THEN CONTINUE

END FOR

END

This algorithm postpones all decision-making until every record has been examined and ignores conflict, battery and security constraints.

A2.1 – Diagnose the Computational & Security Waste | 1 Mark analyse where unnecessary

work, energy drain and potential data leakage occur.

A2.2 – Redesign Using Incremental & Constraint-Aware Synchronisation | 2 Marks

Write pseudocode that constructs the synchronisation set incrementally. At each record

your algorithm should explicitly represent the alternatives: **Upload →**

Defer → Discard / Resolve Conflict

Introduce appropriate pruning based on network quality, battery level, record priority and encryption status.

A2.3 – Demonstrate Pruning | 1 Mark

Use a concrete set of five records with differing priorities, sizes and sensitivity. Show one branch that your algorithm can terminate early. analyse why further exploration of that branch cannot produce a correct or safe synchronisation. **A2.4 – Challenge Your Pruning Rule | 1 Mark**

Suppose offline-first operation with eventual consistency is now required. Determine whether your pruning condition remains valid. If not, identify precisely which assumption has been violated.

Evidence Expected: Analysis of waste, redesigned pseudocode, pruning trace and reasoning under changed requirements.

ANSWER – A2 TASK A2 – Safe Incremental Data Synchronisation (5 Marks) A2.1 – Diagnose Waste

The naïve algorithm attempts every record even when the network is unavailable, wastes radio and CPU energy, retries without a policy, ignores priority and battery level, may send unencrypted sensitive records, and does not detect duplicates or conflicts. Continuing after a failed request also hides the difference between a temporary failure and a permanent security error.

A2.2 – Redesigned Pseudocode

PROCEDURE SYNC_INCREMENTALLY(localRecords, state)\n syncSet ← empty\n FOR each record R in priority order DO\n IF R.isDuplicate OR R.expiredByPolicy THEN DISCARD R; CONTINUE\n IF R.sensitive AND NOT R.encrypted THEN QUARANTINE R; CONTINUE\n IF state.permissionInvalid OR state.sessionInvalid THEN STOP safely\n IF state.networkUnavailable OR state.battery < threshold THEN DEFER R; CONTINUE\n IF remoteVersion(R.id) differs from R.baseVersion THEN\n RESOLVE conflict using version/merge policy; CONTINUE\n END IF\n ADD R to syncSet\n response ← UPLOAD R with idempotency key\n IF response acknowledged THEN mark synced\n ELSE IF retryable THEN defer with exponential backoff\n ELSE quarantine and record reason\n END FOR\nEND PROCEDURE

A2.3 – Demonstrate Pruning

Record	Priority / size / sensitivity	Condition	Decision
R1	Critical / 2 KB / sensitive	Encrypted, Wi-Fi available	Upload
R2	Low / 1 MB / nonsensitive	Battery 8%, threshold 15%	Defer
R3	High / 4 KB / sensitive	Encryption missing	Quarantine; do not explore upload branch
R4	Medium / 20 KB / sensitive	Encrypted, network unavailable	Defer
R5	Low / 3 KB / nonsensitive	Duplicate ID already acknowledged	Discard duplicate

Early termination example: R3 is rejected from the upload branch immediately because it contains sensitive data without encryption. Further network retries cannot make this branch safe; encryption or an approved secure representation must be produced first.

A2.4 – Challenge the Rule
Offline-first eventual consistency changes the assumption that records must be synchronised immediately. Deferring a record because the network is unavailable remains valid, but discarding it merely because it is old or temporarily conflicting may violate eventual consistency. The system must retain durable records, use stable IDs and versions, and resolve conflicts when connectivity returns.

TASK A3 – DISCOVER THE MOBILE-BACKGROUND DESIGN PATTERN *5*

Marks

Consider the following three problems:

Background Sensor Collection → Offline Data Synchronisation → Push Notification Delivery

Complete the following design-analysis table:

Design Component	**Sensor Collection**	**Data Sync**	**Push Notification**
State			
Decision variable			
Candidate choices			
Feasibility / Constraint test			
Goal condition			

Pruning / Deferral condition			

TASK A2 – A2 – Why the Naïve Synchronisation Loop Is Wasteful

Unnecessary radio activation

The naïve SyncAll procedure attempts to upload every local record without first checking whether a usable network is available. A failed connection can cause repeated radio wake-ups, connection establishment attempts and timeouts. Mobile radios consume energy when transitioning between low-power and active states, so repeated short failures can be disproportionately expensive. The algorithm also treats a large record and a small urgent record identically, even though their transfer times and energy requirements are different. A constraint-aware design should inspect the network before opening a transfer and should batch eligible records when batching is safe.

No distinction between failure classes

A network timeout is not equivalent to an authentication failure, an invalid record or a server-side rejection. Continuing the loop after every failure hides these distinctions and may repeat an operation that cannot succeed. A temporary network error should normally defer the record with a retry time. An expired session should stop authenticated transfer until the user signs in again. An invalid or unencrypted sensitive record should be quarantined for correction rather than retried. Classifying failures reduces wasted computation and gives the user or administrator a meaningful explanation.

Privacy and consistency risks

Uploading every local record without checking sensitivity and encryption can expose information. The naïve loop also fails to compare local and remote versions, so concurrent edits may overwrite newer information. Duplicate records can be sent repeatedly if the application restarts after an uncertain response. These are not merely performance problems: they affect confidentiality, integrity and correctness. A safe synchronisation design must decide whether each record can be uploaded, deferred, discarded under an explicit retention rule or sent to a conflict-resolution procedure.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Record selection	Inspect records incrementally and order them by priority and age.	Avoids processing records that cannot currently be sent.
Security gate	Quarantine sensitive records that are not encrypted or authenticated.	A network retry cannot repair a security failure.

Resource gate	Defer work when network quality or battery is below the configured threshold.	Reduces radio wake-ups and unnecessary energy use.
Conflict handling	Compare record versions and apply a documented merge or conflict policy.	Maintains consistency when local and remote copies change.

ENGINEERING TAKEAWAY

Key point: defer temporary failures, quarantine unsafe records and resolve version conflicts before declaring synchronisation complete.

TASK A2 – A2 – Incremental Selection and Safe Pruning

Constructing the synchronisation set

The synchronisation set should be built one record at a time. For each candidate, first validate its identifier and retention status, then check permission and session validity, encryption status, network requirements, battery policy and version compatibility. Only records that pass the required checks should enter the upload set. This incremental construction avoids preparing a large batch that later has to be rejected because one global condition has changed. It also allows high-priority records to be considered before large, low-priority records when the policy permits that ordering.

Upload, defer, discard and conflict branches

The four outcomes represent different meanings. Upload is selected when the record is valid, authorised, encrypted and currently feasible. Defer is selected when the record remains valid but a temporary condition such as weak connectivity or low battery prevents safe transfer. Discard is permitted only when a documented rule proves that the record is redundant, expired or no longer required. Resolve Conflict is selected when local and remote versions have diverged. A conflict must not be silently treated as a duplicate because the two versions may contain different user changes.

Early pruning of unsafe branches

A sensitive record without encryption can be removed from the upload branch immediately. Further network exploration cannot make that transfer safe because connectivity does not provide confidentiality by itself. The record should be quarantined or passed through an approved encryption step before it becomes eligible. Similarly, if the user has logged out and the operation requires the authenticated session, the current upload branch should stop. The local record may remain protected for a later authorised session, subject to the application retention policy.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Record selection	Inspect records incrementally and order them by priority and age.	Avoids processing records that cannot currently be sent.
Security gate	Quarantine sensitive records that are not encrypted or authenticated.	A network retry cannot repair a security failure.
Resource gate	Defer work when network quality or battery is below the configured threshold.	Reduces radio wake-ups and unnecessary energy use.
Conflict handling	Compare record versions and apply a documented merge or conflict policy.	Maintains consistency when local and remote copies change.

ENGINEERING TAKEAWAY

Key point: defer temporary failures, quarantine unsafe records and resolve version conflicts before declaring synchronisation complete.

TASK A2 – A2 – Eventual Consistency and Conflict Handling

Revising assumptions for offline-first operation

An offline-first application is designed to remain useful when the network is absent. Therefore, lack of connectivity should normally cause deferral rather than deletion. The local store becomes the durable source of pending changes, and synchronisation occurs when a suitable connection returns. The earlier assumption that a record must be synchronised immediately is no longer valid. The application must preserve local changes, attach stable identifiers and version metadata, and provide a predictable rule for reconciling changes made on more than one device.

Version checks and conflict resolution

A record can carry a local version, a base version and a server version. Before applying an update, the client compares the base version with the current server version. If they match, the update can usually be applied. If they differ, the client must determine whether the data can be merged, whether one version has priority under a documented rule, or whether the user must resolve the conflict. Last-write-wins can be simple, but it may discard meaningful edits when device clocks are inaccurate. The selected strategy should match the data type and the cost of losing information.

Durable queues and duplicate protection

An offline queue should survive process termination and device restart. Each item should have a stable operation identifier so that the server can recognise a repeated submission. The queue should retain retry count, creation time, last error and next eligible retry time. Old records should not be discarded solely because they have waited a long time unless a stated retention rule permits it. This design supports eventual delivery while still allowing the application to remove genuine duplicates and protect the backend from repeated processing.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Record selection	Inspect records incrementally and order them by priority and age.	Avoids processing records that cannot currently be sent.
Security gate	Quarantine sensitive records that are not encrypted or authenticated.	A network retry cannot repair a security failure.
Resource gate	Defer work when network quality or battery is below the configured threshold.	Reduces radio wake-ups and unnecessary energy use.
Conflict handling	Compare record versions and apply a documented merge or conflict policy.	Maintains consistency when local and remote copies change.

ENGINEERING TAKEAWAY

Key point: defer temporary failures, quarantine unsafe records and resolve version conflicts before declaring synchronisation complete.

Undo / Cancel operation			

A3.1 – Identify the Common Pattern | 2 Marks

Using your completed analysis, analyse what these apparently different mobile problems have in common computationally and architecturally.

A3.2 – Design a Generic Background-Work Framework | 2 Marks Develop general

pseudocode of the form:

SCHEDULE_OR_EXECUTE(State)

Your framework must be sufficiently general to represent all three problems. **A3.3 – analyse**

the Role of Undo / Cancellation | 1 Mark

analyse why a background-work framework must restore or clean up previous state after a cancelled or failed task. Use one of the three problems to illustrate what could happen if this step were omitted.

Evidence Expected: Completed comparison table, generic pseudocode and conceptual explanation.

ANSWER – A3 TASK A3 – Common Mobile Background-Work Pattern (5 Marks)

Design element	Sensor collection	Data synchronisation	Push notification
Component	Sensor API plus foreground service when justified	WorkManager worker / repository	Push messaging service and notification manager
State	Last sample time, permission, sensor status	Queue, retry count, versions, network	Message ID, delivery status, notification permission
Decision variable	Sample now or later	Upload, defer, merge or discard duplicate	Display, defer or suppress
Candidate choices	Collect, batch, pause	Upload, retry, defer, resolve conflict	Notify, group, store silently
Feasibility test	Sensor permission and battery policy	Network, encryption, authentication, battery	Valid payload, app state and notification permission
Goal	Required samples captured	Safe, consistent delivery	Critical information reaches user

Pruning / deferral	Skip redundant samples or batch	Wait for suitable network/battery	Suppress duplicate or invalid notification
Undo / cancel	Stop sensor listener and release resources	Cancel worker; preserve unsynced data	Cancel stale notification or clear pending intent

A3.1 – Common Pattern
All three problems are state-driven background workflows. Each selects a work item, checks constraints, executes or defers it, records outcomes, and handles retries or cancellation. Their common architecture is a scheduler plus state store, feasibility rules, worker, result handler and instrumentation.

A3.2 – Generic Framework

PROCEDURE SCHEDULE_OR_EXECUTE(State)\n update State from OS events\n item ← select eligible item by priority and age\n IF no item exists THEN return WAIT\n IF safety, permission or resource constraints fail THEN defer/cancel item\n ELSE execute item in an appropriate worker\n record start time, resource state and result\n IF success THEN commit state change\n ELSE restore safe state and retry only if permitted\n END IF\n repeat until queue empty or no work is currently feasible

A3.3 – Undo / Cancellation
Cancellation must release resources and restore a consistent state. For example, if a sync worker is cancelled after sending a record but before receiving acknowledgement, it must retain the local record and retry with an idempotency key. Otherwise a record could be lost or duplicated.

TASK A4 – TRACTABILITY: LOCAL vs CLOUD vs HYBRID PROCESSING

Performance, Security and Scalability Classes

4 Marks

Consider the following six mobile-computing problems:

● On-device image classification (tiny model)

● Cloud-based speech recognition

● Peer-to-peer file sharing among nearby devices

● End-to-end encrypted messaging

● Real-time multiplayer game state synchronisation

TASK A3 – A3 – Shared Structure of Background Mobile Work

State-driven execution

Sensor collection, data synchronisation and push-notification delivery appear to be different functions, but each can be represented as a state-driven workflow. The application has a set of pending work items, a current system state and a set of rules that determine whether an item may proceed. The scheduler selects a candidate, checks feasibility, starts an appropriate component, records the result and either commits the outcome or restores a safe state. This common structure allows the application to reuse scheduling, persistence, logging and cancellation mechanisms.

Common component model

A generic background-work framework can contain a durable work queue, a scheduler, a constraint evaluator, a worker interface, a result handler and an instrumentation module. A sensor worker obtains a reading, a synchronisation worker transfers a record and a notification worker presents an alert. The workers differ in their operation, but they can report success, temporary failure, permanent failure or cancellation through a common result model. A common framework reduces duplicated code and makes the behaviour easier to test.

Different policies within one framework

The framework should not force all tasks to use identical constraints. Sensor collection may require sensor permission and a suitable sampling interval. Synchronisation may require encryption, authentication and a network connection. Notification delivery may require a valid message, a notification channel and user permission. The scheduler can call task-specific feasibility rules through a common interface. This preserves the reusable structure without hiding the important differences between task types.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
State	Store item ID, current status, retry count, permissions and resource requirements.	The scheduler can make repeatable decisions.
Decision	Choose execute, defer, retry, cancel or resolve-conflict actions.	Different background tasks use one common control pattern.

Feasibility	Check permissions, battery, network, authentication and OS restrictions.	Only eligible tasks enter execution.
Undo	Release listeners, close resources and preserve unacknowledged data.	Cancellation leaves the application in a safe state.

ENGINEERING TAKEAWAY

Key point: background work is a state-driven process. Every execution path must define success, retry, cancellation and cleanup.

TASK A3 – A3 – Generic Scheduling and State Transitions

Selection and execution sequence

The generic SCHEDULE_OR_EXECUTE(State) procedure begins by refreshing the system state and loading pending work. It selects an item using priority, age and dependency information. It then checks whether the item is still valid and whether its constraints are satisfied. If a temporary constraint fails, the item is deferred with a future eligibility condition. If a permanent safety condition fails, the item is cancelled or quarantined. If the item is eligible, the framework records a start event, invokes the appropriate worker and waits for a structured result.

Commit only after confirmed success

A worker should not update durable application state before the operation has reached a safe commit point. A sensor sample should be stored before it is considered collected. A synchronised record should be marked delivered only after acknowledgement. A notification should be marked displayed only after the notification manager accepts the request. Separating execution from commitment helps the application recover if it is stopped between an operation and its final state update.

Concurrency and dependency management

The framework should limit the number of active workers to protect memory, CPU and battery. Tasks that depend on one another should execute in a defined order. For example, an upload should not begin before its payload has been encrypted and stored. Independent tasks may run concurrently if the resource budget allows it. A scheduler can use a small worker pool and prioritise urgent work while preventing a continuous stream of high-priority tasks from starving ordinary maintenance forever.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
State	Store item ID, current status, retry count, permissions and resource requirements.	The scheduler can make repeatable decisions.
Decision	Choose execute, defer, retry, cancel or resolve-conflict actions.	Different background tasks use one common control pattern.
Feasibility	Check permissions, battery, network, authentication and OS restrictions.	Only eligible tasks enter execution.
Undo	Release listeners, close resources and preserve unacknowledged data.	Cancellation leaves the application in a safe state.

ENGINEERING TAKEAWAY

Key point: background work is a state-driven process. Every execution path must define success, retry, cancellation and cleanup.

TASK A3 – A3 – Cancellation, Undo and Recovery

Why cancellation needs cleanup

Cancellation is not complete merely because a worker stops receiving instructions. The worker may have acquired a sensor listener, opened a file, reserved memory, created a notification or begun a network transaction. It must release resources and restore the application to a consistent state. Cleanup should be safe to execute more than once because a process may be interrupted while handling cancellation. A framework should therefore make cleanup operations idempotent wherever possible.

Example: interrupted synchronisation

Suppose a synchronisation worker sends a record and is cancelled before receiving the server acknowledgement. Deleting the local record at this point would risk data loss because the client does not know whether the server accepted the request. The correct action is to retain the encrypted local

record and retry later using the same idempotency key. If the server already accepted the first request, it can recognise the repeated identifier and avoid creating a duplicate.

Example: stopped sensor collection

If a sensor worker is cancelled, it should unregister its listener, close any open resource and save any completed reading that has already reached the durable store. If listener cleanup is omitted, the application may continue receiving events after the task is considered stopped. This can waste battery and create duplicate readings. The generic framework should therefore define a cleanup callback and record whether the cleanup operation completed.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
State	Store item ID, current status, retry count, permissions and resource requirements.	The scheduler can make repeatable decisions.
Decision	Choose execute, defer, retry, cancel or resolve-conflict actions.	Different background tasks use one common control pattern.
Feasibility	Check permissions, battery, network, authentication and OS restrictions.	Only eligible tasks enter execution.
Undo	Release listeners, close resources and preserve unacknowledged data.	Cancellation leaves the application in a safe state.

ENGINEERING TAKEAWAY

Key point: background work is a state-driven process. Every execution path must define success, retry, cancellation and cleanup.

● Large-scale offline map rendering with vector tiles

For each problem prepare a concise analysis containing:

Problem	**What constitutes a solution?**	**Can a proposed design be efficiently verified?**	**Complexity / Resource class**	**Reason**
1				
2				

3				
4				
5				
6				

Use the resource classes: On-device feasible | Network-bound | Battery-critical | Security-critical | Hybrid required

Inquiry

Why is “runs slowly on a phone” not an adequate definition of a performance problem in mobile computing?

and

Why is the distinction between local processing and cloud offloading important when interpreting scalability and security of a mobile application?

Evidence Expected: Correct classification accompanied by reasoning about verification, resource constraints and problem formulation.

ANSWER – A4

TASK A4 – Local vs Cloud vs Hybrid Processing (4 Marks)

Problem	What constitutes a solution?	Efficiently verified?	Resource class and reason
On-device image classification	Return a class label and confidence for an input image	Yes: compare predictions with labelled test data	On-device feasible; tiny model avoids network delay and protects image privacy.
Cloud speech recognition	Send permitted audio securely and return recognised text	Yes: compare word error rate and latency	Network-bound; quality can be high but depends on connectivity, privacy and server load.
Peer-to-peer file sharing	Transfer verified files between nearby devices	Yes: hashes and transfer logs verify integrity	Hybrid required; local discovery/transfer saves internet use but requires authentication and interruption recovery.
End-to-end encrypted messaging	Only intended endpoints can decrypt messages	Partly: protocol tests verify properties, but formal/security review is also needed	Security-critical; keys, identity verification and secure storage dominate.
Real-time multiplayer synchronisation	Keep player states acceptably consistent within latency bounds	Yes: measure latency, jitter, packet loss and divergence	Hybrid required; local prediction plus server authority balances responsiveness and consistency.
Offline vector map rendering	Render map tiles smoothly without network access	Yes: compare rendered output and frame times	Battery-critical / ondevice feasible; caching speeds access but increases storage and memory use.

Why “runs slowly on a phone” is insufficient: performance must be defined with measurable limits such as frame time, latency, memory, energy, throughput and workload. A delay may come from CPU computation, network transfer, disk access, contention or battery throttling.

Why local/cloud distinction matters: local processing avoids network latency and can keep data on the device, but consumes device CPU, memory and battery. Cloud processing can scale server resources, but depends on connectivity, adds transmission cost and exposes data to additional trust boundaries. Hybrid designs select placement per task.

Implement the greedy / adaptive strategy designed in Task A5.

TASK A4 – A4 – Resource Classes and Placement Decisions

Meaning of on-device processing

On-device processing keeps computation on the mobile device. It is appropriate when the input is small enough, the model or algorithm fits the memory budget, response time must be short, and data should remain local. Tiny image classification is a representative example because a compact model can process an image without sending it to a server. The design must still account for CPU or accelerator availability, thermal throttling, battery state and model-update requirements. Local execution avoids network latency but does not make computation free.

Meaning of cloud processing

Cloud processing sends data to a remote service for computation. It is useful when the task requires a large model, extensive storage or resources that are not available on typical mobile devices. Cloud speech recognition is an example when the application relies on a server-side recognition engine. The design must include network delay, service availability, data-transfer cost, authentication and privacy protection. A cloud solution can be computationally powerful but may become unavailable in weak-connectivity environments.

Meaning of hybrid processing

Hybrid processing divides work between the device and a remote system. The device may perform filtering, compression, feature extraction or local caching before sending a smaller representation for more expensive processing. Offline maps can use locally cached vector tiles while downloading missing regions when connectivity returns. Hybrid design is useful when responsiveness and offline operation are both important, but it requires clear rules about which component owns each state and how results are reconciled.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Local processing	Run small and privacy-sensitive work on the device when resources allow.	Avoids network delay but consumes device battery and memory.
Cloud processing	Offload compute-heavy work when connectivity and privacy rules permit.	Uses server capacity but adds network and trust-boundary costs.

Hybrid placement	Split discovery, preprocessing, heavy computation and final validation across locations.	Balances latency, scalability and device constraints.
Verification	Measure latency, throughput, memory, energy, integrity and security properties.	A performance claim needs measurable acceptance criteria.

ENGINEERING TAKEAWAY

Key point: choose local, cloud or hybrid placement using measurable latency, energy, memory, connectivity and security requirements.

TASK A4 – A4 – Verification, Complexity and Security

What constitutes an efficient solution

A solution is not efficient only because it completes the task. It must complete the required function within acceptable limits for time, memory, energy, network use and storage. Verification asks whether the proposed design can be checked against correctness and safety properties. For example, a peer-to-peer file transfer should verify peer identity, file integrity and transfer completion. A messaging system should verify that message confidentiality and authentication are preserved across retries and device changes.

Complexity and resource class

The analysis should state the main computational cost and the dominant resource constraint. Image classification may be dominated by model inference and memory. Speech recognition may be dominated by network round-trip time and server computation. Peer-to-peer sharing may be dominated by discovery, connection setup and transfer bandwidth. Multiplayer state synchronisation may be dominated by update frequency, participant count and latency. Offline map rendering may be dominated by tile decoding, caching and graphics memory. The resource class should follow from these characteristics rather than from a vague label.

Security as a placement constraint

Processing location affects privacy and attack surface. Keeping data on-device can reduce exposure during transmission, but a compromised device may still reveal local data. Cloud processing requires protected transport, server authentication, access control and retention rules. Peer-to-peer communication requires secure peer discovery and verification. End-to-end encrypted messaging must prevent an intermediary from reading message content.

Security should therefore be evaluated alongside performance, not added after the placement decision.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Local processing	Run small and privacy-sensitive work on the device when resources allow.	Avoids network delay but consumes device battery and memory.
Cloud processing	Offload compute-heavy work when connectivity and privacy rules permit.	Uses server capacity but adds network and trust-boundary costs.
Hybrid placement	Split discovery, preprocessing, heavy computation and final validation across locations.	Balances latency, scalability and device constraints.
Verification	Measure latency, throughput, memory, energy, integrity and security properties.	A performance claim needs measurable acceptance criteria.

ENGINEERING TAKEAWAY

Key point: choose local, cloud or hybrid placement using measurable latency, energy, memory, connectivity and security requirements.

TASK A4 – A4 – Applying the Classification to the Six Problems

Image classification and speech recognition

A tiny image-classification model is generally an on-device candidate when the model fits the available memory and the expected accuracy is sufficient. The design can be tested using a fixed validation set and measured inference time. Cloud speech recognition is a cloud-oriented candidate when the application depends on a larger recognition model, but it should provide a local fallback or deferred mode if the network is absent. The report should compare latency, privacy, accuracy and energy rather than assuming one placement is universally superior.

Peer-to-peer sharing and encrypted messaging

Peer-to-peer file sharing is naturally distributed because nearby devices exchange data directly. Its main challenges include discovering peers, authenticating them, managing interrupted transfers and verifying file integrity. End-to-end encrypted messaging may use a hybrid architecture: devices perform encryption and decryption locally, while a server may relay ciphertext and delivery

metadata. The server should not possess the keys required to read message content. This separation supports reachability without giving the relay unrestricted access to private communication.

Multiplayer synchronisation and offline maps

Real-time multiplayer games need frequent state updates and low latency. A hybrid model may keep immediate prediction and rendering on the device while using a server as the authority for shared game state. Offline map rendering uses locally cached tiles for immediate display and downloads missing tiles when suitable connectivity is available. Both systems must manage stale data, limited bandwidth and cache eviction. Their resource class depends on update rate, geographic or participant scale, memory budget and the tolerance for temporary inconsistency.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Local processing	Run small and privacy-sensitive work on the device when resources allow.	Avoids network delay but consumes device battery and memory.
Cloud processing	Offload compute-heavy work when connectivity and privacy rules permit.	Uses server capacity but adds network and trust-boundary costs.
Hybrid placement	Split discovery, preprocessing, heavy computation and final validation across locations.	Balances latency, scalability and device constraints.
Verification	Measure latency, throughput, memory, energy, integrity and security properties.	A performance claim needs measurable acceptance criteria.

ENGINEERING TAKEAWAY

Key point: choose local, cloud or hybrid placement using measurable latency, energy, memory, connectivity and security requirements.

B3.1 – Correctness on Small Instances | 2 Marks

Construct at least three small test instances. For each report Exact cost, Approximate cost, and whether they match.

Do not deliberately choose only instances where the heuristic matches the optimum. **B3.2 –**

Find a Counter-example | 2 Marks

Construct an instance for which your adaptive strategy does not return the minimum cost configuration. Trace the adaptive choices and identify the decision that led away from the optimum.

B3.3 – Increase the Problem Size | 2 Marks

Increase the number of feed items and quality levels. Instrument both approaches. Record meaningful operation counts (configurations evaluated, quality changes, cache evictions) rather than only wall-clock time.

B3.4 – Engineering Trade-off | 2 Marks

Using your experimental evidence, answer: What did the approximate solution gain computationally, and what did it sacrifice in solution quality or guarantee? Identify a realistic situation in which you would accept the approximate solution.

Evidence Expected: Two implementations / simulations, three normal cases, student designed counter-example, scalability evidence and engineering interpretation.

ANSWER – A5 TASK A5 – Approximate UI / Graphics Optimisation (4 Marks) A5.1 – Exact Perspective

An exhaustive method enumerates every combination of decoding resolution, cache size and prefetch distance, measures estimated frame drops and energy, then chooses the lowest-cost feasible combination. If there are q quality levels, c cache sizes and p prefetch distances, there are q × c × p configurations per device profile. Testing many profiles and feed states quickly becomes expensive and can delay adaptation.

A5.2 – Adaptive Heuristic

PROCEDURE ADAPT_FEED(metrics, config, battery)\n REPEAT\n measure frameTime, droppedFrames, memoryUse and battery\n IF frameTime > 16 ms OR memoryUse > limit THEN\n reduce decode quality one step; evict least-recently-used offscreen item\n shorten prefetch window\n ELSE IF frameTime < 12 ms AND battery is healthy AND memory has headroom THEN\n increase quality one step or extend prefetch

TASK A5 – A5 – UI Approximation and Performance Objectives

Why UI performance matters

A mobile interface is judged by how quickly it responds to touch, scroll and navigation. A technically correct screen can still be unusable if it drops frames, blocks the main thread or consumes excessive battery. A common display target is a frame interval of about 16.7 milliseconds for a 60 Hz display, although actual refresh rates vary by device. The application should measure frame time and missed frames rather than relying only on visual impressions. Performance should be evaluated on representative devices, including low-memory devices and devices under thermal or battery constraints.

Exact and approximate configuration choices

UI quality, cache size and prefetch distance interact. Higher image quality may increase decoding cost and memory use. A larger cache can reduce repeated decoding but consumes memory. Longer prefetch can make scrolling smoother but may load content that the user never views. An exact search can enumerate a small number of configurations and select the feasible configuration with the lowest stated cost. An approximate strategy changes settings in response to current measurements and is more suitable when the decision must be made quickly.

Cost function and constraints

The cost function in the assignment combines dropped frames, energy units and memory penalty. The coefficients represent the relative importance assigned to each outcome. The application should reject any configuration that exceeds a hard frame-time or memory limit before comparing costs. A low numerical cost is not acceptable if the configuration violates a safety or usability constraint. The selected configuration should be reported together with its measured or simulated frame time, memory use and energy estimate.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Frame-time target	Track frame time and dropped frames during scrolling and rendering.	Identifies visible responsiveness problems.
Quality control	Reduce decoding quality gradually when frame time or memory exceeds limits.	Protects interaction without abrupt configuration changes.

Cache policy	Evict least-recently-used off-screen items when memory pressure rises.	Keeps frequently needed content available.
Prefetch policy	Extend or shorten prefetch distance according to battery and observed performance.	Avoids unnecessary downloads and memory use.

ENGINEERING TAKEAWAY

Key point: adaptive optimisation improves responsiveness quickly, but it does not guarantee the globally minimum-cost configuration.

TASK A5 – A5 – Adaptive Quality, Caching and Prefetch

Starting configuration

An adaptive strategy can begin with medium visual quality, a moderate cache and a short prefetch distance. This avoids starting with the most expensive configuration on every device. The system then observes frame time, memory pressure and battery state over a suitable measurement window. If performance is poor, it can reduce image quality or shorten prefetch. If performance is stable and resources are available, it can cautiously increase quality.
The strategy should change one major parameter at a time so that the effect of each change can be identified.

Cache replacement policy

A least-recently-used policy removes items that have not been accessed recently when the cache reaches its memory limit. This is a practical heuristic because recently viewed items are often more likely to be revisited during scrolling. However, LRU is not always optimal. A large item that is rarely used may occupy memory, while a small item needed repeatedly may be evicted under a poor access pattern. The cache should therefore track item size and decoding cost when the application has enough information to improve the decision.

Prefetch control

Prefetching attempts to load content before the user requests it. A short prefetch distance reduces unnecessary network and decoding work, while a longer distance may reduce visible loading delays during fast scrolling. The correct distance depends on scroll speed, network latency, content size and available memory. The application should cancel obsolete prefetch requests when the user changes direction or leaves the screen. Otherwise, work that no longer contributes to the visible interface may waste energy and bandwidth.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Frame-time target	Track frame time and dropped frames during scrolling and rendering.	Identifies visible responsiveness problems.
Quality control	Reduce decoding quality gradually when frame time or memory exceeds limits.	Protects interaction without abrupt configuration changes.
Cache policy	Evict least-recently-used off-screen items when memory pressure rises.	Keeps frequently needed content available.
Prefetch policy	Extend or shorten prefetch distance according to battery and observed performance.	Avoids unnecessary downloads and memory use.

ENGINEERING TAKEAWAY

Key point: adaptive optimisation improves responsiveness quickly, but it does not guarantee the globally minimum-cost configuration.

TASK A5 – A5 – Evaluation and Limitations of Approximation

Comparing the three test instances

The assignment provides three illustrative cases: a steady Wi-Fi feed, a low-memory device and a flagship device with stable frame time. The approximate method matches the exact result in two cases and differs in the low-memory case. This difference is important because it demonstrates that a heuristic is not guaranteed to reproduce the global optimum. The evaluation should explain which resource observation caused the adaptive method to choose a different configuration and whether the resulting cost remains acceptable for the intended user experience.

Counter-example and local decisions

A local adjustment may appear beneficial but can prevent the strategy from reaching a better combination of settings. For example, a brief frame-time spike may cause quality to be reduced, after which the algorithm increases cache size. The new combination may have a higher total cost than a configuration that keeps higher quality with a smaller cache. This counter-example shows that interacting variables cannot always be optimised independently. A practical adaptive system can

periodically compare nearby configurations or return to a known baseline when performance remains stable.

Operational safeguards

Approximation should be bounded by minimum quality and maximum resource limits. The application should avoid rapid oscillation between settings by using hysteresis, smoothing and a minimum interval between major changes. It should retain a safe fallback configuration for devices that repeatedly exceed frame-time or memory limits. The system should also record why a setting changed so that developers can identify whether the policy is responding to genuine resource pressure or noisy measurements. These safeguards improve stability without claiming a global optimality guarantee.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Frame-time target	Track frame time and dropped frames during scrolling and rendering.	Identifies visible responsiveness problems.
Quality control	Reduce decoding quality gradually when frame time or memory exceeds limits.	Protects interaction without abrupt configuration changes.
Cache policy	Evict least-recently-used off-screen items when memory pressure rises.	Keeps frequently needed content available.
Prefetch policy	Extend or shorten prefetch distance according to battery and observed performance.	Avoids unnecessary downloads and memory use.

ENGINEERING TAKEAWAY

Key point: adaptive optimisation improves responsiveness quickly, but it does not guarantee the globally minimum-cost configuration.

window\n END IF\n avoid changing more than one major parameter per evaluation\n update config and record the reason for each change\n UNTIL frameTime remains within target for several windows OR no safe adjustment remains\n RETURN config

A5.3 – Trade-off

A heuristic can find a feasible smooth-scrolling configuration quickly, but it does not prove that the total energy cost is globally minimal for a given visual quality. It may settle on a local choice or react to short-term measurements. It is acceptable for real-time feeds because responsiveness matters and conditions change continuously; periodic measurement and safe limits reduce risk.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

The following compact simulations provide reproducible example logic and illustrative tables. Run equivalent code on your chosen device/emulator and replace illustrative values with measured evidence for final submission.

Use your background-work implementation from Task B1.

Construct two workloads having the same number of records / events and using the same constraint thresholds.

Workload A – Design an instance that your algorithm resolves with relatively little deferral or cancellation.

Workload B – Design an instance intended to cause substantially more deferrals, retries or constraint violations.

Before Running – Predict which workload will require more work, where you expect unsuccessful attempts to arise, and why pruning / deferral is likely to occur earlier or later.

Then execute both and compare: work items processed, deferrals, successful deliveries.

Investigation – analyse why input size alone fails to describe the actual computational and energy effort required by a mobile background pipeline on a particular instance. Identify the structural property of your difficult workload that caused additional work.

Evidence Expected: Two student-designed workloads, prediction, measured behaviour and interpretation.

ANSWER – B1 TASK B1 – Instrumented Background Architecture (7 Marks) B1.1 – Implementation Model and Tests

Each work item has an ID, priority, encrypted flag, network requirement and status. The scheduler logs enqueue, defer, cancel and success events. A unique ID prevents duplicate scheduling. Workers are bounded to a small concurrency limit, and retryable failures use backoff.

Test	Input condition	Expected behaviour	What it verifies

Good connectivity	Battery 80%, Wi-Fi available, permission valid	Eligible encrypted items upload and are acknowledged	Normal successful execution
Doze / low battery	Battery 10%, noncritical task, Doze active	Task is deferred and retained	Constraint-aware scheduling and battery policy
Permission revoked	Permission becomes invalid during worker	Worker stops safely; no new sensor read/upload	Boundary handling and privacy

B1.2 – Design vs Implementation

The pseudocode describes the architecture independently of language. In implementation, the OS may stop a worker or process unexpectedly, so state must be persisted before execution and each upload must be idempotent. These changes make retries safe after process death.

B1.3 – Two Scheduling Strategies

Version 1 (FIFO) selects the oldest queued task. Version 2 uses a score: score = 5 × priority + 2 × ageBonus + criticalFlag, subject to permission, encryption, network and battery constraints. Critical work receives an earlier opportunity, but unsafe items are never promoted past a security check.

B1.4 – Illustrative Measurement

Measure	Version 1: FIFO	Version 2: priority heuristic
Work items enqueued	10	10
Deferred / cancelled	4	4
Successful executions	6	6
Peak concurrent workers	2	2
Critical records delivered first	2 of 4	4 of 4

Interpretation: Both strategies obey the same constraints and complete the same number of eligible items. The priority strategy improves delivery order for critical records, but it does not reduce the number deferred in this example. Different workloads may produce different results.

TASK B1 – B1 – Instrumentation Architecture and Counters

Events to record

The implementation should create a trace event whenever a work item is enqueued, selected, started, deferred, cancelled, completed or quarantined. Each event should include a stable task identifier, timestamp, work type and reason code. At a decision point, record the battery level, network type and availability, Doze state, permission status and active worker count. The trace should avoid recording sensitive payloads or authentication tokens. These events provide enough information to reconstruct why the scheduler made a particular decision without exposing user data.

Concurrency measurement

Peak concurrent workers is the largest number of workers active at the same time during a test. It can be measured by incrementing an active-worker counter when a worker starts and decrementing it when the worker finishes or is cancelled. The maximum observed value is retained for the final report. A concurrency limit prevents too many workers from competing for CPU, memory and radio resources. The implementation should verify that the observed peak never exceeds the configured limit, including when retries are scheduled.

Backlog and completion accounting

At the end of each test, the implementation should report how many items were delivered, how many remain queued, how many were deferred and how many were cancelled. These categories should be mutually understandable and should not double-count a single item as both delivered and pending. A deferred item is not a failure if the required constraint is temporarily unavailable. The final outcome should state whether all eligible work completed or whether a residual backlog remains and why.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Instrumentation	Record enqueue, start, defer, cancel, success, retry and acknowledgement events.	Makes the scheduler's behaviour measurable.
Constraints	Log battery, network, Doze, permission and authentication state at each decision.	Explains why an item was executed or deferred.

Concurrency	Apply a fixed worker limit and record peak simultaneous workers.	Prevents uncontrolled resource contention.
Validation	Test healthy connectivity, low battery/Doze and permission revocation.	Covers normal, constrained and boundary conditions.

ENGINEERING TAKEAWAY

Key point: counters and decision-state logs turn implementation behaviour into evidence that can be compared with the design.

TASK B1 – B1 – Validation Scenarios and Expected Behaviour

Good connectivity and healthy battery

The first test should provide stable Wi-Fi, a battery level above the configured threshold, valid permissions and an authenticated session. Eligible records should be selected and uploaded. The test verifies that the normal execution path works, that successful acknowledgements update the local state and that the queue eventually becomes empty. It should also check that duplicate work identifiers do not create duplicate uploads and that the worker count remains within the configured limit.

Doze or low-battery condition

The second test should place the device in Doze or set the simulated battery below the threshold for non-critical work. The expected behaviour is to defer ordinary uploads and retain encrypted local records. The test verifies that the scheduler respects operating-system constraints instead of repeatedly attempting work that cannot proceed. If a critical item is permitted to use a different path, the implementation should document the policy and still apply authentication, permission and privacy checks.

Permission revoked during execution

The boundary test should revoke the relevant permission after a worker has started but before it completes. The worker should detect the change at a safe checkpoint, stop further collection or transfer, release resources and record a cancellation reason. The test verifies that the implementation does not rely only on permission status captured when the task was enqueued. It should also verify that already stored records follow the stated retention and authorisation policy.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result

Instrumentation	Record enqueue, start, defer, cancel, success, retry and acknowledgement events.	Makes the scheduler's behaviour measurable.
Constraints	Log battery, network, Doze, permission and authentication state at each decision.	Explains why an item was executed or deferred.
Concurrency	Apply a fixed worker limit and record peak simultaneous workers.	Prevents uncontrolled resource contention.
Validation	Test healthy connectivity, low battery/Doze and permission revocation.	Covers normal, constrained and boundary conditions.

ENGINEERING TAKEAWAY

Key point: counters and decision-state logs turn implementation behaviour into evidence that can be compared with the design.

TASK B1 – B1 – FIFO and Priority Scheduling Comparison

FIFO scheduling

FIFO processes work in arrival order. It is simple to implement and provides predictable ordering, but it does not distinguish a critical event from a routine maintenance record. A large low-priority item at the front of the queue can delay a small urgent item. FIFO is useful as a baseline because its behaviour is easy to reproduce and compare. The test should record completion order, deferred count, successful executions and peak concurrency under the same workload used for the priority strategy.

Priority heuristic

A priority heuristic can rank items using urgency, age, size and current feasibility. Critical alerts may receive a higher rank, while older routine items can gradually gain priority to prevent starvation. The selection rule should not allow priority to override permission, encryption or authentication constraints. A possible score is a weighted combination of urgency and waiting time, with a separate eligibility filter applied before ranking. The weights are design assumptions and should be stated clearly.

Interpreting the comparison

The priority strategy is useful if it improves delivery time for critical records without increasing unsafe retries or leaving ordinary records permanently deferred. A lower average completion time does not automatically mean the entire system is better; the report should also inspect residual backlog and fairness. Both strategies must use identical input workloads and constraint states. Any difference should be attributed to the ordering rule rather than to different test conditions.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Instrumentation	Record enqueue, start, defer, cancel, success, retry and acknowledgement events.	Makes the scheduler's behaviour measurable.
Constraints	Log battery, network, Doze, permission and authentication state at each decision.	Explains why an item was executed or deferred.
Concurrency	Apply a fixed worker limit and record peak simultaneous workers.	Prevents uncontrolled resource contention.
Validation	Test healthy connectivity, low battery/Doze and permission revocation.	Covers normal, constrained and boundary conditions.

ENGINEERING TAKEAWAY

Key point: counters and decision-state logs turn implementation behaviour into evidence that can be compared with the design.

TASK B2 – ENGINEER AN ADVERSARIAL WORKLOAD

Same Size, Very Different Behaviour

5 Marks

ANSWER – B2 TASK B2 – Adversarial Workload (5 Marks)

Property	**Workload A: easy**	**Workload B: difficult**
Number of records	8	8
Network	Stable Wi-Fi	Repeated disconnects / weak signal
Battery	70–90%	8–18%
Security state	All sensitive records encrypted	Some records unencrypted or expired session
Priority mix	Few high-priority records	Several critical records plus low-priority large records
Prediction	Most eligible records succeed in first attempt	More deferrals, retries and quarantines

Illustrative result	**Workload A**	**Workload B**
Items processed	8	8
Deferrals	1	5
Successful deliveries	7	3
Security quarantines	0	2

Input size alone is not a reliable measure of computational or energy effort. Workload B has the same record count but more network failures, constraint checks, retries and security rejections. Its structure—not just size—causes additional processing and radio wake-ups.

TASK B3 – EXACT vs APPROXIMATE UI / GRAPHICS OPTIMISATION *8*

Marks

ANSWER – B3 TASK B3 – Exact vs Approximate UI Optimisation (8 Marks)

Solution A: Exact Search

For a small instance, enumerate each quality/cache/prefetch configuration, calculate cost = 4 × droppedFrames + energyUnits + 2 × memoryPenalty, reject configurations exceeding the frame or memory limit, and return the feasible configuration with minimum cost.

TASK B2 – B2 – Designing Workloads with Different Computational Cost

Why item count is insufficient

Two workloads can contain the same number of records but require very different amounts of computation and energy. Record size affects transfer time, while sensitivity checks, encryption, validation and conflict resolution add CPU work. Network failures introduce retries and radio wake-ups. A workload with a few large low-priority records may therefore consume more resources than a workload with many small records that can be delivered immediately.
The experiment should describe workload structure, not merely count the number of items.

Workload A: mostly eligible records

Workload A contains a small number of records with valid permissions, active authentication and suitable network conditions. Most records are expected to succeed on the first attempt. The illustrative result in the assignment shows eight items processed, one deferral, seven successful deliveries and no security quarantines. This workload represents a favourable operating condition and provides a baseline for comparing the scheduler and instrumentation.

Workload B: constrained and adversarial records

Workload B contains the same number of records but includes weak connectivity, low-priority large payloads, retryable failures and sensitive records that fail an encryption check. The illustrative result shows more deferrals and fewer successful deliveries, together with security quarantines. The result demonstrates that workload composition changes computational effort even when the record count is unchanged. The values are illustrative simulation results and should not be represented as measurements from a physical device.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Workload structure	Keep item count constant while varying network, battery and security conditions.	Isolates the effect of workload structure.
Failure injection	Introduce disconnects, weak signal, expired sessions and retryable errors.	Reveals the cost of recovery paths.

Expected counters	Compare processed items, deferrals, retries, quarantines and successful deliveries.	Shows more than elapsed time alone.
Interpretation	Relate extra work to repeated checks, radio activation and recovery.	Explains why equal input sizes can have different costs.

ENGINEERING TAKEAWAY

Key point: equal item counts do not imply equal effort. Failures, retries, constraints and security checks determine actual cost.

TASK B2 – B2 – Adversarial Conditions and Failure Paths

Network interruption at an unfavourable time

An adversarial workload can interrupt connectivity after a request is sent but before the acknowledgement arrives. The client cannot safely assume that the server rejected the record. It should retain the local copy and retry with the same idempotency key. This scenario tests whether the implementation can tolerate uncertain outcomes without losing data or creating duplicate records. The trace should distinguish a timeout from a confirmed server rejection.

Permission and authentication changes

Another adversarial condition revokes permission or expires authentication while work is queued or running. The scheduler should stop the affected operation and record the reason. It should not repeatedly retry a request that cannot become valid until the user takes action. The test should verify that the application does not expose sensitive details in its logs and does not continue using credentials from an expired session.

Resource pressure and starvation

A workload can also create memory pressure by placing several large records in the queue while a high-priority stream arrives continuously. If the scheduler always selects the highest priority item, low-priority work may never run. Age-based priority or a reserved service share can reduce starvation. The experiment should measure waiting time by priority class and inspect whether the queue grows without bound. A scheduler should provide predictable behaviour under sustained load, not only under a short favourable test.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Workload structure	Keep item count constant while varying network, battery and security conditions.	Isolates the effect of workload structure.
Failure injection	Introduce disconnects, weak signal, expired sessions and retryable errors.	Reveals the cost of recovery paths.
Expected counters	Compare processed items, deferrals, retries, quarantines and successful deliveries.	Shows more than elapsed time alone.
Interpretation	Relate extra work to repeated checks, radio activation and recovery.	Explains why equal input sizes can have different costs.

ENGINEERING TAKEAWAY

Key point: equal item counts do not imply equal effort. Failures, retries, constraints and security checks determine actual cost.

TASK B2 – B2 – Interpreting Results and Improving the Experiment

Use consistent test conditions

Workload comparisons are meaningful only when both scheduling versions receive the same input records, network trace, battery trace and permission events. The random seed should be fixed if the simulation includes random failures. The report should state whether values are measured, simulated or estimated. This distinction is important because illustrative values can explain a method but cannot establish real-world performance without an implementation and repeatable experiment.

Metrics beyond successful count

Successful delivery count should be considered alongside average and maximum queue age, number of retries, bytes transferred, worker execution time, peak concurrency and energy estimate. A strategy that delivers more records by using excessive retries may not be efficient. A strategy that

defers many records may preserve battery but leave a large backlog. The correct interpretation depends on the application objective and the importance of timely delivery.

Improvement based on evidence

If the experiment shows repeated failures for large records on weak networks, the system may split uploads into resumable chunks or defer them until a better connection is available. If low-priority items starve, the priority rule can include waiting time. If concurrency causes memory pressure, the worker limit can be reduced. Each change should address an observed failure mode and be tested again using the same baseline workload so that the effect can be isolated.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Workload structure	Keep item count constant while varying network, battery and security conditions.	Isolates the effect of workload structure.
Failure injection	Introduce disconnects, weak signal, expired sessions and retryable errors.	Reveals the cost of recovery paths.
Expected counters	Compare processed items, deferrals, retries, quarantines and successful deliveries.	Shows more than elapsed time alone.
Interpretation	Relate extra work to repeated checks, radio activation and recovery.	Explains why equal input sizes can have different costs.

ENGINEERING TAKEAWAY

Key point: equal item counts do not imply equal effort. Failures, retries, constraints and security checks determine actual cost.

Solution B: Adaptive Strategy
Start with medium quality, moderate cache and short prefetch. If frame time exceeds 16 ms, reduce quality and evict the least-recently-used item. If frame time is comfortably below target and battery is healthy, cautiously increase quality or prefetch.

B3.1 – Three Small Test Instances

Instance	Exact minimum cost	Approximate cost	Match?
1: steady Wi-Fi, moderate feed	18	18	Yes
2: low-memory device	12	14	No
3: flagship, stable frame time	10	10	Yes

These are illustrative simulation values, not device measurements. The table deliberately includes a mismatch to avoid assuming the heuristic always matches the optimum.

B3.2 – Counter-example
Suppose high quality with a small cache has total cost 9, while medium quality with a larger cache has cost 11. The adaptive strategy observes one brief frame-time spike, reduces quality, and then increases cache size to avoid future misses. Its local reaction selects cost 11 and does not return to the globally better configuration. The decision is locally reasonable but misses the interaction between quality and cache.

B3.3 – Increasing Problem Size

Problem size	Exact configurations evaluated	Adaptive quality changes	Adaptive cache evictions
3 items, 2 quality levels	8	2	1
10 items, 4 quality levels	64	4	5
50 items, 6 quality levels	216	6	17

The counts are illustrative and should be regenerated from the submitted implementation. Exact search evaluates every configuration; adaptive search evaluates a small number of changes based on current measurements.

B3.4 – Engineering Trade-off
Approximation reduces search work and responds quickly to changing frame time, but it sacrifices a global optimality guarantee and can choose a higher-cost configuration. It is suitable for continuous scrolling, where a slightly suboptimal configuration is preferable to visible stalls while exhaustive search runs.

TASK B4 – FINAL ENGINEERING DECISION

Multimedia Framework Selection under Production Constraints *5 Marks*

A production mobile application must capture, encode and upload short video clips from thousands of concurrent users. Each device has different camera hardware, CPU, battery state and network conditions. A placement / encoding decision is required quickly.

B4.1 – Implement a Practical Strategy | 2 Marks

Implement a reasonable heuristic strategy for choosing resolution, bitrate and upload timing. State the selection rule used. Output must show chosen parameters, estimated energy / time and remaining battery impact for a set of sample devices.

B4.2 – Challenge Your Strategy | 1 Mark

Construct an input (device profile + network) intended to make your strategy choose a suboptimal configuration, or analyse through comparison with a small exhaustive search why the heuristic may fail to be optimal.

B4.3 – Production Decision Note | 2 Marks

Prepare a short Algorithm / Architecture Engineering Decision Note addressing:

- Why exhaustive search over encoding parameters is unattractive at production scale
- What guarantee your chosen strategy does or does not provide ● What computational / energy advantage it provides
- When an exact or cloud-assisted approach might still be appropriate ● What evidence you would monitor before deploying the strategy

Evidence Expected: Working implementation or detailed simulation, allocation / parameter output, challenging input and evidence-based engineering decision.

TASK B3 – B3 – Exact Search Formulation

Configuration space

The exact method represents each candidate interface configuration as a combination of visual quality, cache capacity and prefetch distance. For each combination, the algorithm estimates dropped frames, energy use and memory penalty. It calculates the stated objective cost and rejects configurations that violate the frame-time or memory constraint. Among the remaining candidates, it selects the configuration with the smallest cost. This method is suitable for small search spaces because every feasible combination is considered.

Correctness of exhaustive evaluation

An exact search is optimal only with respect to the defined candidate set, cost function and constraints. If the cost function omits an important factor, such as thermal throttling or user-perceived image quality, the selected result may not be best in practice. The report should state the assumptions used to calculate each cost term. It should also verify that every candidate is evaluated once and that infeasible candidates are not accidentally selected.

Complexity growth

If there are q quality levels, c cache settings and p prefetch settings, the number of combinations is $q \times c \times p$. Adding more parameters increases the search space multiplicatively. Exact search can be practical for a small offline experiment but may be too slow to repeat during continuous scrolling. The implementation should report the number of configurations evaluated and the time required, rather than describing the method only as expensive.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Exact search	Enumerate each quality, cache and prefetch configuration for small instances.	Provides a reference minimum when the search space is manageable.
Adaptive rule	Adjust one major parameter at a time using measured frame time and memory.	Reduces decision overhead during live interaction.

Counter-example	Compare the heuristic choice with the best feasible configuration.	Demonstrates that local decisions may miss a global optimum.
Evaluation	Count configurations, quality changes, evictions and frame drops.	Makes the computational trade-off reproducible.

ENGINEERING TAKEAWAY

Key point: exact search is a useful reference for small cases; adaptive methods trade optimality guarantees for faster decisions.

TASK B3 – B3 – Adaptive Heuristic and Stability

Feedback-based adjustment

The adaptive strategy observes current frame time and resource state, then changes a limited number of settings. If frame time exceeds the target, it can reduce visual quality or shorten prefetch. If memory pressure rises, it can evict cache entries. If the device has adequate battery and stable performance, it can cautiously increase quality. This approach reacts to current conditions and avoids evaluating every possible combination.

Avoiding oscillation

A noisy frame-time measurement can cause repeated quality changes if the strategy reacts immediately to every sample. The application should smooth measurements over a short window and use separate thresholds for degrading and restoring quality. This hysteresis prevents rapid switching around one boundary. It should also impose a minimum interval between major changes and retain a known safe configuration when resource conditions become unstable.

Limits of local optimisation

The heuristic may choose a locally sensible adjustment that is not globally optimal. A brief frame-time spike may trigger a quality reduction, after which a cache increase is selected. The resulting combination may have a higher total cost than another configuration that was never considered. The assignment counter-example illustrates this limitation. The heuristic should therefore be described as responsive and resource-aware, not as a guaranteed optimum.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Exact search	Enumerate each quality, cache and prefetch configuration for small instances.	Provides a reference minimum when the search space is manageable.
Adaptive rule	Adjust one major parameter at a time using measured frame time and memory.	Reduces decision overhead during live interaction.
Counter-example	Compare the heuristic choice with the best feasible configuration.	Demonstrates that local decisions may miss a global optimum.
Evaluation	Count configurations, quality changes, evictions and frame drops.	Makes the computational trade-off reproducible.

ENGINEERING TAKEAWAY

Key point: exact search is a useful reference for small cases; adaptive methods trade optimality guarantees for faster decisions.

TASK B3 – B3 – Test Results, Scaling and Engineering Trade-off

Interpreting the three test instances

The assignment reports exact and approximate costs of 18 and 18 for the steady Wi-Fi case, 12 and 14 for the low-memory case, and 10 and 10 for the stable flagship case. The matching cases show that the heuristic can reach the exact result for some conditions. The low-memory mismatch shows that the heuristic can select a feasible but higher-cost configuration. These values are illustrative and should be regenerated from the submitted implementation before being described as experimental measurements.

Scaling comparison

The exact method evaluates all combinations in the defined search space, while the adaptive method evaluates a small number of changes based on current observations. As the number of content items and quality levels grows, exhaustive evaluation can become increasingly costly. The heuristic

reduces decision work, but its quality depends on the rules, thresholds and measurements. A larger search space does not automatically make a heuristic accurate; it only makes exhaustive evaluation more expensive.

Choosing the appropriate method

Exact search is appropriate when the configuration space is small, the decision is made offline or the cost of a suboptimal result is high. Approximate adaptation is appropriate when conditions change quickly and a decision must be made within a short time. A production system may combine both methods by computing a safe set of configurations offline and selecting among them at runtime. This approach preserves fast response while limiting the choices available to the adaptive controller.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Exact search	Enumerate each quality, cache and prefetch configuration for small instances.	Provides a reference minimum when the search space is manageable.
Adaptive rule	Adjust one major parameter at a time using measured frame time and memory.	Reduces decision overhead during live interaction.
Counter-example	Compare the heuristic choice with the best feasible configuration.	Demonstrates that local decisions may miss a global optimum.
Evaluation	Count configurations, quality changes, evictions and frame drops.	Makes the computational trade-off reproducible.

ENGINEERING TAKEAWAY

Key point: exact search is a useful reference for small cases; adaptive methods trade optimality guarantees for faster decisions.

MARK DISTRIBUTION

Component	**Task**	**Marks**
Part A – Design & Reasoning	A1 – Intent/Service Architecture	7
	A2 – Sync & Pruning	5
	A3 – Background Design Pattern	5
	A4 – Resource Classes	4
	A5 – UI Approximation Design	4
Part A Total		**25**
Part B – Implementation	B1 – Instrumented Background	7
	B2 – Adversarial Workload	5
	B3 – Exact vs Approximate UI	8
	B4 – Multimedia Decision	5
Part B Total		**25**

TOTAL		**50**

SUBMISSION REQUIREMENTS For Part A, submit the required problem formulation, pseudocode / architectural diagrams, traces, comparison tables and reasoning. Programming-language code will not be accepted as pseudocode.

For Part B, submit source code together with test inputs, expected behaviour, actual outputs, instrumentation data and interpretation. A screenshot showing only a final answer is not sufficient evidence.

Students are expected to use the same engineering workflow throughout the assignment:

UNDERSTAND → DESIGN → CONSTRAINT-CHECK → IMPLEMENT → MEASURE → CHALLENGE → INTERPRET

The key question is not simply "Did the application work?" It is also:

How much background work, energy and network traffic was required, why was it required, when does the design stop being practical, and what should an engineer do about it?

ANSWER – B4 TASK B4 – Final Engineering Decision: Multimedia Framework (5 Marks) B4.1 – Practical Encoding Heuristic

Selection rule: if battery is below 20% or network is weak, choose 480p at low bitrate and defer upload until a stable connection; if battery is at least 40% and Wi-Fi is stable, choose 1080p at medium/high bitrate; otherwise choose 720p at medium bitrate. Always encrypt the clip before queueing and upload only after authentication.

Device profile	Battery / network	Chosen settings	Upload decision	Illustrative estimate
A	80%, Wi-Fi	1080p, 4 Mbps	Upload now	2 min; 3% battery
B	35%, LTE fair	720p, 2 Mbps	Upload when stable	3 min; 4% battery
C	15%, weak LTE	480p, 0.8 Mbps	Defer until WiFi/charging	1 min encode; 1% battery
D	55%, Wi-Fi, hot device	720p, 1.5 Mbps	Wait for temperature to fall	2 min; 2% battery

B4.2 – Challenge the Strategy

A device with 18% battery but an urgent emergency clip over excellent Wi-Fi may be assigned 480p and deferred by the heuristic, even though a higher-quality immediate upload may be

required. This is a counter-example because the rule values battery over urgency. A production system should include a user/business priority flag and an emergency override with explicit consent and energy limits.

B4.3 – Production Decision Note

- Exhaustive search over resolution, bitrate, codec, upload timing and device profiles is expensive at thousands of concurrent users and may not finish before the decision is needed.
- The heuristic guarantees that selected settings follow stated thresholds, but it does not guarantee globally minimum energy or maximum quality.
- It reduces computation by evaluating a small number of rules and reduces energy through lower bitrate, batching and deferred uploads.
- Exact or cloud-assisted optimisation may be appropriate for offline planning, high-value clips, or devices with sufficient resources and stable connectivity; urgent cases may use a priority-aware policy.
- Before deployment monitor encode time, upload latency, failure/retry count, battery drain, temperature, video quality, privacy/security errors and completion rate.

OVERALL CONCLUSION

The assignment demonstrates that mobile applications need constraint-aware scheduling, secure incremental synchronisation, explicit background-work state, and measurable UI/multimedia optimisation. Exact search can provide an optimum for small spaces, while adaptive heuristics reduce decision cost for dynamic mobile conditions. A reliable design must preserve encrypted data during failures, respect permissions and OS limits, instrument real behaviour, test adverse cases, and clearly state where a heuristic sacrifices optimality.

TASK B4 – B4 – Multimedia Pipeline and Platform Frameworks

Capture, encode and upload stages

A mobile multimedia pipeline contains several stages: camera capture, frame buffering, encoding, encrypted local storage, upload scheduling and server acknowledgement. Each stage has a different resource profile. Capture and encoding may use the camera hardware and media codec, while upload depends on network quality and authentication. The application should avoid holding full-resolution frames in memory longer than necessary. It should release buffers after the encoder no longer needs them and should preserve a recoverable local clip until upload completion is confirmed.

Android and iOS framework responsibilities

On Android, the application can use the platform camera and media APIs to capture and encode video, while WorkManager or an appropriate foreground service controls background transfer according to the operation and platform rules. On iOS, Core Media represents media timing and sample buffers, while Core Video manages image-buffer and pixel-buffer resources used in video processing. These frameworks provide platform-specific building blocks; the application still needs to manage resource ownership, permissions, background execution and failure recovery.

Resolution and bitrate selection

Resolution determines the spatial detail of the clip, while bitrate controls the approximate amount of encoded data produced per second. Higher resolution and bitrate can improve visual quality but increase encoding work, storage requirements, transfer time and battery use. A practical policy selects a setting that matches device capability, battery state, thermal condition and network quality. The chosen parameters should be recorded so that the result can be reproduced and compared across devices.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Capture	Read camera frames using supported platform APIs and validate permissions.	Creates a reliable input to the multimedia pipeline.
Encode	Select resolution and bitrate using battery, temperature and network state.	Balances quality, processing time and energy.

Upload	Encrypt and queue clips; upload after authentication and suitable connectivity.	Protects data and avoids repeated failed transfers.
Production monitoring	Track encode time, upload latency, retries, battery drain and completion rate.	Provides evidence for deployment decisions.

ENGINEERING TAKEAWAY

Key point: multimedia settings must balance quality, battery, thermal state, network conditions, privacy and completion time.

TASK B4 – B4 – Practical Heuristic and Device Profiles

Rule-based selection

The assignment uses a simple rule: when battery is below 20 percent or the network is weak, select 480p at a low bitrate and defer upload until a stable connection; when battery is at least 40 percent and Wi-Fi is stable, select 1080p at a medium or high bitrate; otherwise select 720p at a medium bitrate. A hot device should avoid a demanding encoding profile until temperature falls. The rule is easy to explain and inexpensive to execute, but its thresholds are design assumptions that should be validated on target devices.

Device profile evaluation

For each sample device, the implementation should report battery level, network type, thermal state, selected resolution, bitrate, upload decision, estimated encoding time and estimated battery impact. The source assignment provides illustrative profiles: a high-battery Wi-Fi device selects 1080p; a medium-battery fair-LTE device selects 720p; a low-battery weak-LTE device selects 480p and defers upload; and a hot device uses a moderate setting while waiting for temperature to fall. These estimates are examples, not measurements from actual phones.

Encryption and authentication

The clip should be encrypted before it is placed in the upload queue, and upload should occur only after the user session and destination are authenticated. The application should not store access tokens in plain text or include private media data in diagnostic logs. If authentication expires while the clip is queued, the file can remain protected locally until a valid session is available, subject to the retention policy. A transfer retry should use a stable clip identifier to avoid duplicate server-side processing.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Capture	Read camera frames using supported platform APIs and validate permissions.	Creates a reliable input to the multimedia pipeline.
Encode	Select resolution and bitrate using battery, temperature and network state.	Balances quality, processing time and energy.
Upload	Encrypt and queue clips; upload after authentication and suitable connectivity.	Protects data and avoids repeated failed transfers.
Production monitoring	Track encode time, upload latency, retries, battery drain and completion rate.	Provides evidence for deployment decisions.

ENGINEERING TAKEAWAY

Key point: multimedia settings must balance quality, battery, thermal state, network conditions, privacy and completion time.

TASK B4 – B4 – Challenging the Heuristic and Production Decision

Counter-example involving urgency

A device with 18 percent battery and excellent Wi-Fi may need to upload an urgent emergency clip immediately. The simple heuristic would choose low resolution and defer the upload because battery is below 20 percent. That decision may be suboptimal because it ignores the value and urgency of the clip. A production policy should include a documented priority class, explicit user or application consent and an energy limit for emergency handling. An override must not bypass encryption or authentication.

Why exhaustive search is unattractive

A full search over resolution, bitrate, codec, frame rate, upload timing and device-specific constraints can create a large configuration space. Repeating that search for thousands of concurrent users would consume computation and delay decisions. A rule-based heuristic evaluates a small number of conditions and can make a decision quickly. Its advantage is lower decision overhead and

predictable execution time, not a guarantee that every device receives the globally optimal media configuration.

Guarantees and monitoring

The heuristic guarantees only that its stated rules are followed when the inputs are valid. It does not guarantee minimum energy, maximum quality or minimum upload time across all possible configurations. Before deployment, the engineering team should monitor encoding duration, upload latency, retry count, battery drain, device temperature, clip quality, completion rate and security failures. The policy should be revised when measured results show that a threshold is too aggressive or causes repeated deferrals.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Capture	Read camera frames using supported platform APIs and validate permissions.	Creates a reliable input to the multimedia pipeline.
Encode	Select resolution and bitrate using battery, temperature and network state.	Balances quality, processing time and energy.
Upload	Encrypt and queue clips; upload after authentication and suitable connectivity.	Protects data and avoids repeated failed transfers.
Production monitoring	Track encode time, upload latency, retries, battery drain and completion rate.	Provides evidence for deployment decisions.

ENGINEERING TAKEAWAY

Key point: multimedia settings must balance quality, battery, thermal state, network conditions, privacy and completion time.

TASK B4 – B4 – Deployment, Scalability and Final Evaluation

When exact or cloud-assisted optimisation is useful

Exact optimisation may be appropriate during offline testing, for a small number of device classes or when the application must meet a strict quality target. A cloud-assisted service may recommend profiles based on aggregated device capabilities, but the mobile client must still make safe decisions

when disconnected. The server should not require continuous telemetry that exposes unnecessary personal information. A hybrid design can distribute validated profiles to devices and let each device choose among them using current battery, temperature and network measurements.

Scalability under concurrent users

At production scale, the backend must handle bursts of uploads without allowing every device to retry at the same time. Upload scheduling should use backoff and jitter, and the server may apply rate limits or admission control. Clips can be uploaded in resumable chunks so that a temporary interruption does not require restarting the entire transfer. The application should also enforce storage limits and a retention policy so that deferred clips do not consume unlimited device storage.

Overall engineering conclusion

The multimedia decision should balance visual quality, energy, thermal state, network availability, privacy and delivery urgency. The rule-based strategy is a practical starting point because it is understandable and inexpensive, but the counter-example shows why production policies need priority awareness and measured thresholds. The final decision should be supported by repeatable test profiles, instrumented outputs and an explicit statement of limitations. A successful engineering design explains not only which parameters were selected, but also why the selection remains safe when the device or network changes.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Capture	Read camera frames using supported platform APIs and validate permissions.	Creates a reliable input to the multimedia pipeline.
Encode	Select resolution and bitrate using battery, temperature and network state.	Balances quality, processing time and energy.
Upload	Encrypt and queue clips; upload after authentication and suitable connectivity.	Protects data and avoids repeated failed transfers.
Production monitoring	Track encode time, upload latency, retries, battery drain and completion rate.	Provides evidence for deployment decisions.

ENGINEERING TAKEAWAY

Key point: multimedia settings must balance quality, battery, thermal state, network conditions, privacy and completion time.

TASK B4 – B4 – Final Quality Assurance and Evidence Plan

Pre-deployment validation

Before release, the multimedia pipeline should be tested on devices representing low, medium and high processing capability. Each test should capture the selected resolution, bitrate, codec, encoding duration, peak memory use, temperature change and battery impact. Network tests should include stable Wi-Fi, weak mobile data, temporary disconnection and a connection that returns after a long delay. The same sample clips should be used across devices so that quality and resource differences can be compared fairly.

Failure recovery and data integrity

The application should be tested when the operating system stops the process during encoding, when the upload is interrupted and when authentication expires. A partially encoded clip should not be treated as complete. A partially uploaded clip should resume from a verified offset or restart safely using a stable identifier. The local encrypted copy should remain available until the server confirms completion. These checks demonstrate that performance optimisation does not weaken data integrity.

Evidence-based deployment decision

The final deployment decision should compare the measured results with the application requirements. If the low-battery profile causes excessive deferral, the priority rule may need adjustment. If high bitrate produces unacceptable upload delay, the profile may be reduced for mobile networks. If temperature rises too quickly, encoding may be split into smaller jobs or postponed. Every adjustment should be supported by recorded evidence and should retain the security requirements established in the earlier tasks.

DESIGN CHECKPOINTS AND IMPLEMENTATION SUMMARY

Engineering element	Application in this task	Expected result
Capture	Read camera frames using supported platform APIs and validate permissions.	Creates a reliable input to the multimedia pipeline.

Encode	Select resolution and bitrate using battery, temperature and network state.	Balances quality, processing time and energy.
Upload	Encrypt and queue clips; upload after authentication and suitable connectivity.	Protects data and avoids repeated failed transfers.
Production monitoring	Track encode time, upload latency, retries, battery drain and completion rate.	Provides evidence for deployment decisions.

ENGINEERING TAKEAWAY

Key point: multimedia settings must balance quality, battery, thermal state, network conditions, privacy and completion time.